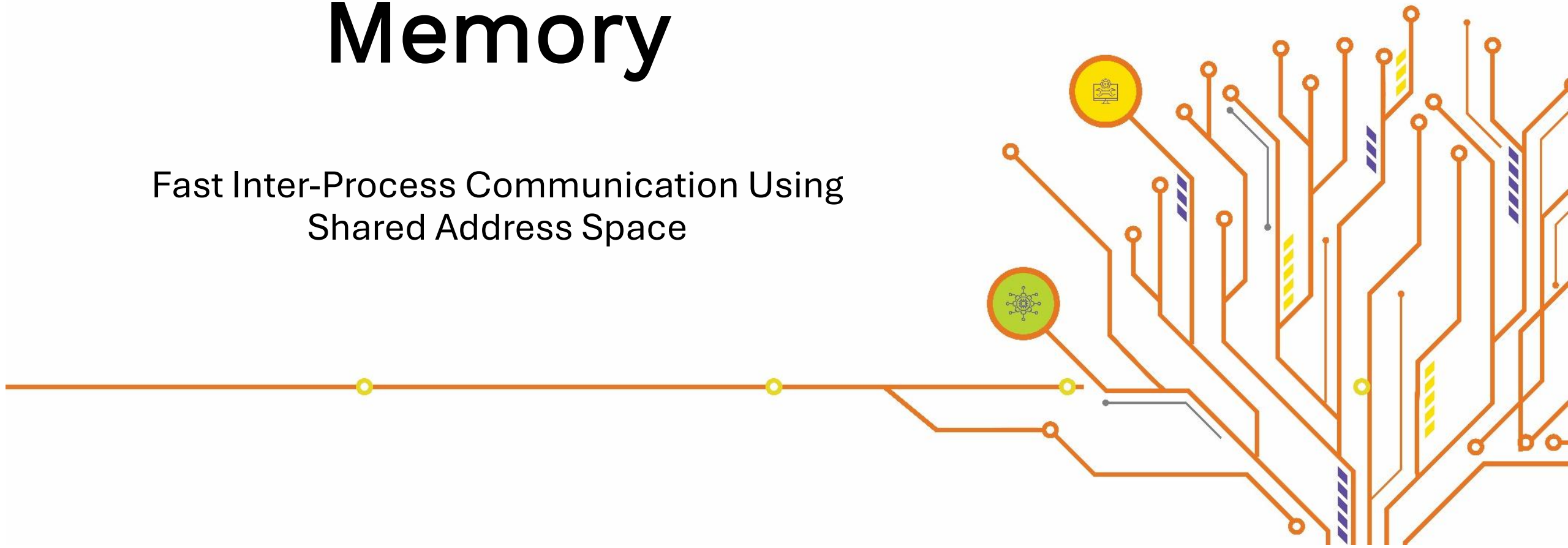
KPIT | APEX LAB

POSIX Shared Memory

Fast Inter-Process Communication Using Shared Address Space



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

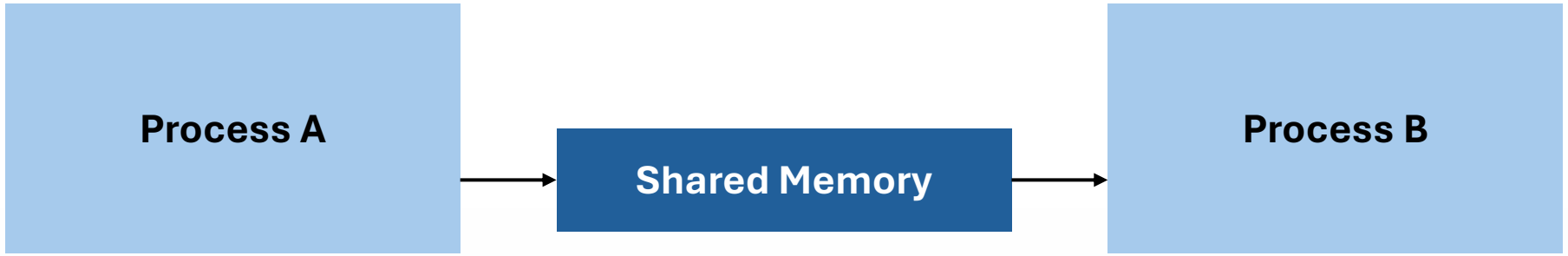
It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

What is Shared Memory?

- Fastest IPC mechanism
- Multiple processes **map the same memory region**
- Data is exchanged by **direct read/write**, not system calls
- Created using **shm_open() + mmap()**
- Shared region lives inside **kernel-managed memory objects**



Why Use Shared Memory?

Highest throughput IPC

Zero-copy → no kernel copying overhead

Ideal for large data transfers

Great for:

- sensor buffers
- image frames
- circular queues
- control blocks
- shared lookup tables

POSIX Shared Memory Basics

- Identified by a **name** beginning with /

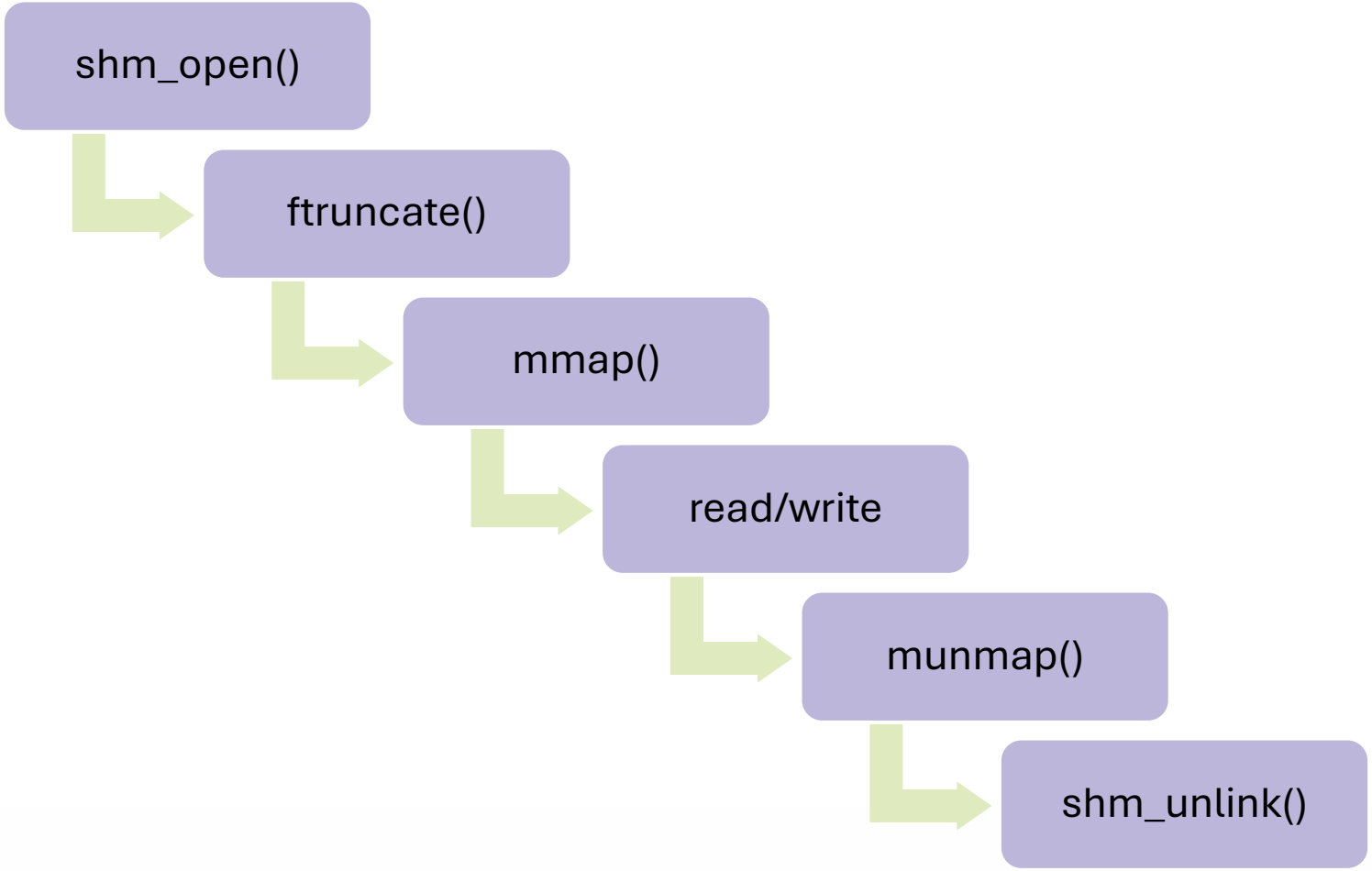
  Example: /apex_shm

- Behaves like a **file descriptor**
- Must specify size using ftruncate()
- Mapped into virtual memory using mmap()
- Must be cleaned using shm_unlink()

Key APIs

- **Create/Open:**
  - int fd = shm_open("/apex", O_CREAT | O_RDWR, 0666);
- **Set Size:**
  - ftruncate(fd, SIZE);
- **Map Memory:**
  - void *ptr = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
- **Cleanup:**
  - munmap(ptr, SIZE);
  - close(fd);
  - shm_unlink("/apex");.

Lifecycle of Shared Memory Object



Mapping Shared Memory

- mmap() parameters:
  - PROT_READ, PROT_WRITE
  - MAP_SHARED (changes visible to all processes)
- Important:
  - Use MAP_SHARED (not MAP_PRIVATE) for IPC
  - Mapping returns pointer → use it like a normal C array/struct

Example: Writer and Reader Process

```
int fd = shm_open("/apex", O_CREAT | O_RDWR, 0666);
ftruncate(fd, 1024);

char *p = mmap(NULL, 1024, PROT_READ|PROT_WRITE,
MAP_SHARED, fd, 0);
sprintf(p, "Hello from writer!");
```

```
int fd = shm_open("/apex", O_RDONLY, 0666);
char *p = mmap(NULL, 1024, PROT_READ, MAP_SHARED,
fd, 0);

printf("Reader got: %s\n", p);
```

Synchronization Required

Shared memory gives fast sharing but introduces race conditions.

- Examples requiring mutex/sem:
  - Updating shared counters
  - Writing multi-field structures
  - Producer-consumer buffers
  - Logging queues

Shared memory ≠ thread-safe.

Using Shared Memory with Mutex

Steps:
1. Put mutex structure **inside** shared memory
2. Initialize with `PTHREAD_PROCESS_SHARED` attribute
3. Multiple processes can lock/unlock it

Diagram:
[Shared Memory] → struct { mutex; data; }

Unmapping & Cleaning Up

- munmap(ptr, size) → detach mapping
- close(fd) → release descriptor
- shm_unlink(name) → delete shared-memory object
- Important:
  - unlink should be done by creator or cleanup tool.

Benefits vs Other IPC

IPC	Speed	Direction	Payload	Best Use
Pipes	Medium	One-way	Byte stream	Parent-child text data
Message Queues	Medium	Two-way	Structured msgs	Events, priorities
Shared Memory	**Fastest**	Both ways	Any data	Large data, real-time

Example 1: Basic Shared Memory (Writer → Reader)

Task:

- Writer creates `/lab_shm`, writes a string
- Reader opens same shared memory and prints it
- Cleanup using `shm_unlink()`

Example 2: Shared Integer Counter

Task:

- Writer initializes a counter to 0
- Reader process increments every second
- Writer prints current value every second
- Observe shared updates

Learning: shared variables without copying

Example 3: Producer–Consumer Buffer

Shared memory contains:

- circular buffer (array)
- two indexes: head, tail
- two semaphores (empty, full)

Implement: Producer writes items; consumer prints them.

Learning: synchronization + shared data structure

Real-World Usa Cases

- Shared vision/image buffer between processes
- Sensor data accumulation shared with control loop
- Shared configuration block for real-time apps
- Logging service shared across modules
- Robotics (shared state, map, telemetry)

Common Pitfalls

- Forgetting ftruncate() → 0-size mapping
- Using MAP_PRIVATE → changes not shared
- Not synchronizing writes → corrupted structures
- Forgetting shm_unlink() → orphaned objects remain
- Writing beyond allocated size → segmentation faults
- Multiple writers without locks → race conditions

Troubleshooting

- Use ls -l /dev/shm to list shared memory objects
- Use ipcs -m (System V, optional for comparison)
- Check return values of all system calls
- Use strace for debugging open/mmap calls
- Use hexdump to inspect raw memory if needed

Summary

- Fastest inter-process communication method
- shm_open + mmap create shared memory region
- Multiple processes map same address space → read/write directly
- Requires explicit synchronization
- Ideal for high-performance embedded systems

Scenario: Telemetry Ring Buffer with Dual Semaphores (1/4)

Context

- Implement a high-rate telemetry pipeline where a producer process samples data (e.g., random sensor values) and writes into a shared memory circular buffer; a consumer process reads, computes rolling statistics (min/max/avg), and prints them.
- Synchronization via counting semaphores empty and full (+ a mutex for head/tail if you choose).

Scenario: Telemetry Ring Buffer with Dual Semaphores (2/4)

PART 1 — Shared Region & Metadata

- Create telemetry_common.h, producer.c, consumer.c.
- Shared memory object /apex_shm_telemetry, size enough for:
  - struct header { int head, tail; int capacity; }
  - struct sample { uint64_t ts_ns; float value; } buf[CAP];
- Initialize with ftruncate and mmap in producer (creator).

Scenario: Telemetry Ring Buffer with Dual Semaphores (3/4)

PART 2 — Synchronization

- Semaphores in shared memory or named semaphores:
    - empty = CAP, full = 0.
- Producer flow (loop @ 100 Hz):
    - sem_wait(empty) → write sample at head → advance head (mod CAP) → sem_post(full)
- Consumer flow (loop):
    - sem_wait(full) → read sample at tail → advance tail → sem_post(empty)
    - Update sliding window stats and print every second.

Scenario: Telemetry Ring Buffer with Dual Semaphores (4/4)

PART 3 — Graceful Shutdown

- Handle SIGINT: producer writes a shutdown flag in header; consumer exits when queue drained.
- Cleanup: munmap, close, shm_unlink (producer), sem_close/sem_unlink if named.